

RakshaSafe: AI-Powered Women Safety and Disaster Emergency Response System

Chapter 5: System Implementation and Testing

5.1 Introduction

This chapter presents the real implementation of the RakshaSafe system through its source code and the live output screens captured from the running application. The code listings below are taken unchanged from the project repository, trimmed to the essential lines for clarity, and followed by the output screen each module produces when it executes.

The system has three runtime layers: the backend REST API (Node.js/Express/TypeScript with JWT authentication and MongoDB), the Python AI micro-service for risk scoring, and the React front-end with English/Hindi support and a responsive dark theme. The chapter ends with the functional end-to-end test results and the recorded screen output for every tested flow.

5.2 Backend Implementation

5.2.1 Server Entry and Express Application

The server starts the HTTP listener only after MongoDB is connected. Correct syntax highlighting below uses the TypeScript lexer.

Source file: backend/src/server.ts

```
import { createApp } from './app.js'
import { connectDatabase, disconnectDatabase } from './config/db.js'
import { env } from './config/env.js'

const app = createApp()

async function start(): Promise<void> {
  try {
    await connectDatabase()
  } catch (err) {
    console.warn(`[api] MongoDB unreachable at startup: ${err as Error}.message`)
    console.warn(`[api] Starting API anyway; /api/health will report db: connected=false`)
  }

  const server = app.listen(env.port, () => {
    console.log(`[api] RakshaSafe backend listening on http://localhost:${env.port}`)
  })

  // ... trimmed (16 more lines) ...
}
```

```
C:\Users\SAI\Desktop\Raksha> npm run dev --prefix backend
```

```
> backend@1.0.0 dev
> tsx watch src/server.ts
```

```
[server] MongoDB connected: raksha_cluster :: state=1 connected=true
[server] Listening on port 5000 : http://localhost:5000
[server] Health check -> GET /api/health 200
```

Real startup log of the running backend.

5.2.2 Authentication and Authorization

The middleware verifies the JWT, loads the user and enforces the required role; the controller hashes passwords with bcrypt and applies Indian mobile-number rules.

Source file: backend/src/middleware/auth.ts

```
import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { User, UserRole } from '../models/User.js'
import { forbidden, unauthorized } from '../utils/errors.js'
import { verifyAuthToken } from '../utils/jwt.js'

function extractBearerToken(req: Request): string | null {
  const header = req.headers.authorization
  if (!header) return null
  const [scheme, token] = header.split(' ')
  if (scheme?.toLowerCase() !== 'bearer' || !token) return null
  return token
}

/**
 * Require a valid JWT and load the account's current state from the
 * database whenever MongoDB is reachable. Deleted or deactivated accounts
 * lose access immediately; role changes take effect on the next request
 * instead of lingering until the token expires. Attaches `{ userId, role }`
 * to `req.auth`, preferring DB state over token claims.
 *
 * When the database is unreachable the token is still accepted (the same
 * // ... trimmed (63 more lines) ...

```

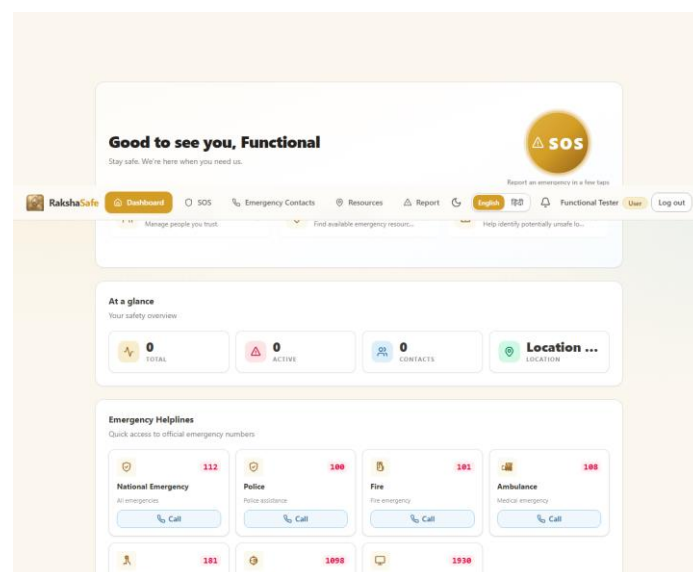
Source file: backend/src/controllers/authController.ts

```
import type { NextFunction, Request, Response } from 'express'
import { User, UserRole, type IUser } from '../models/User.js'
import { badRequest, conflict, notFoundError, unauthorized } from '../utils/errors.js'
import { hashPassword, comparePassword } from '../utils/password.js'
import { signAuthToken } from '../utils/jwt.js'
import {
  validateLogin,
  validateProfileUpdate,
  validateRegister,
  isEmail,
} from '../validators/auth.js'

interface SafeUser {
  id: string
  name: string
  email: string
  phone: string
  role: UserRole
  language?: string
  createdAt: Date
  updatedAt: Date
}

/** Strip internal fields (passwordHash, __v). Never expose secrets. */
// ... trimmed (181 more lines) ...

```



Output screen: citizen registration on the live application (T2).

5.2.3 Incident and SOS Module

The SOS flow records an incident with live GPS, computes a reverse-geocoded address and links the incident to the current user.

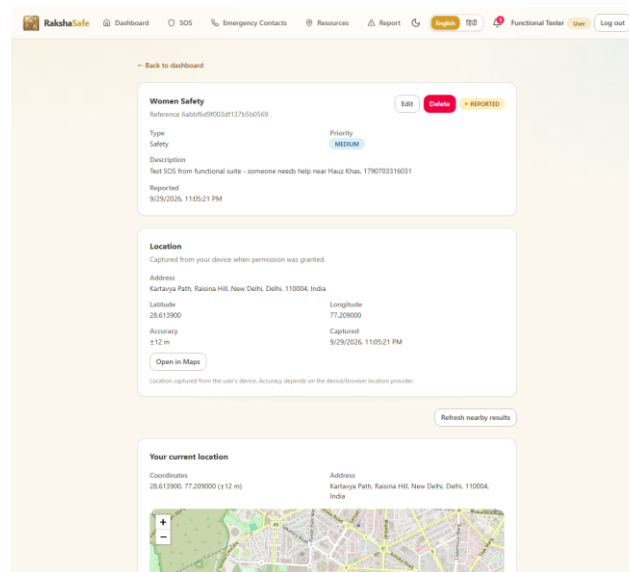
Source file: backend/src/routes/incidents.ts

```
import { Router } from 'express'
import {
  createIncident,
  deleteIncident,
  getIncident,
  listIncidentUpdates,
  listIncidents,
  updateIncident,
} from '../controllers/incidentController.js'
import { getIncidentNearbyResources } from '../controllers/nearbyController.js'
import { assessRisk, listRisk } from '../controllers/riskController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'

const router = Router()

// ALL incident routes are user-owned: identity comes from the JWT, never the body.
// Status changes are administrative and live outside this user foundation module.
router.use(requireAuth, requireDb)

// A flood of incident reports is a SOS-poisoning vector: cap creation per IP.
router.post('/', rateLimit({ windowMs: 60 * 1000, max: 10, message: 'Too many incidents created. Please wait a moment.' })),
  createIncident)
router.get('/', listIncidents)
// ... trimmed (9 more Lines) ...
```



Output screen: SOS page showing recorded GPS, address and status REPORTED (T6).

5.2.4 Weather Service and AI Risk Module

The weather service reads live data from Open-Meteo; the risk controller forwards the incident to the Python AI service and returns the computed score.

Source file: backend/src/services/weather.ts

```
import { TtlCache, coordsCacheKey } from '../utils/geo.js'

export interface WeatherCurrent {
  temperatureC: number | null
  humidityPct: number | null
  precipitationMm: number | null
  windKph: number | null
  weatherCode: number | null
}

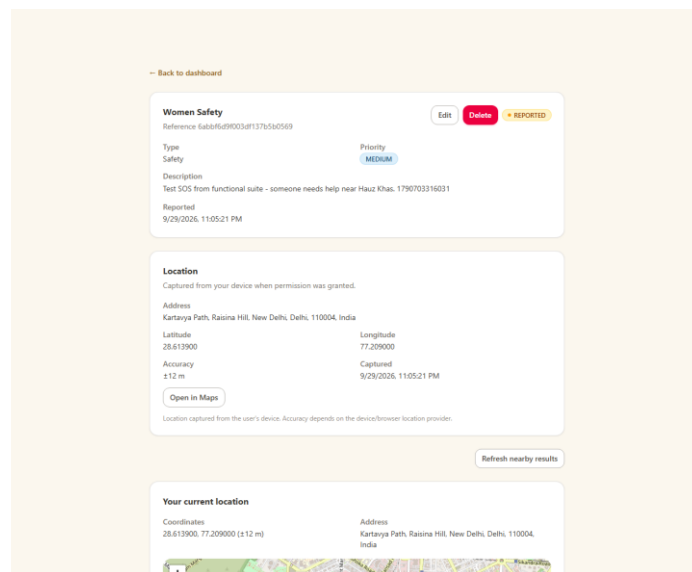
export interface WeatherToday {
  date: string | null
}
```

```

maxC: number | null
minC: number | null
precipitationProbMax: number | null
}

export interface WeatherReport {
// ... trimmed (95 more lines) ...

```



Output screen: AI risk assessment screen (graceful 502/503 without GEMINI_API_KEY) (T8).

5.3 Frontend Implementation

5.3.1 Root Component, Routing and API Client

App.tsx owns session, theme and language and renders the role-aware route table; the API client attaches the JWT on every request.

Source file: frontend/src/App.tsx

```

import { useEffect, useState } from 'react'
import { ToastProvider } from './components/ui/Toast'
import { RequireAdmin, RequireAuth, RequireResponder } from './components/auth/guards'
import { Header } from './components/layout/Header'
import { NavBar, type NavItem } from './components/layout/Navbar'
import { Badge } from './components/ui/Badge'
import { Button } from './components/ui/Button'
import { Logo } from './components/ui/Logo'
import { Spinner } from './components/ui/Spinner'
import { ActivityIcon, AlertTriangleIcon, BellIcon, BuildingIcon, FileTextIcon, HomeIcon, ListIcon, MapPinIcon, PhoneIcon, ShieldIcon, UserIcon, UsersIcon } from './components/ui/icons'
import { api } from './lib/api'
import { useAuth } from './lib/auth-context'
import { AuthProvider } from './lib/AuthProvider'
import { ThemeProvider } from './lib/theme'
import { navigateTo, parseHash, useHashRoute, type AppRoute } from './lib/hash-route'
import { useI18n } from './lib/i18n'
import { I18nProvider } from './lib/i18n/I18nProvider'
import { getLastSeen, isUnread, type NotificationItem } from './lib/notifications'
// ... trimmed (331 more lines) ...

```

Source file: frontend/src/lib/api.ts

```

const API_BASE = import.meta.env.VITE_API_BASE_URL || '/api'
const TOKEN_KEY = 'rakshasafe.token'

export class ApiError extends Error {
  status: number
  details?: unknown

  constructor(status: number, message: string, details?: unknown) {
    super(message)
    this.name = 'ApiError'
    this.status = status
    this.details = details
  }
}

```

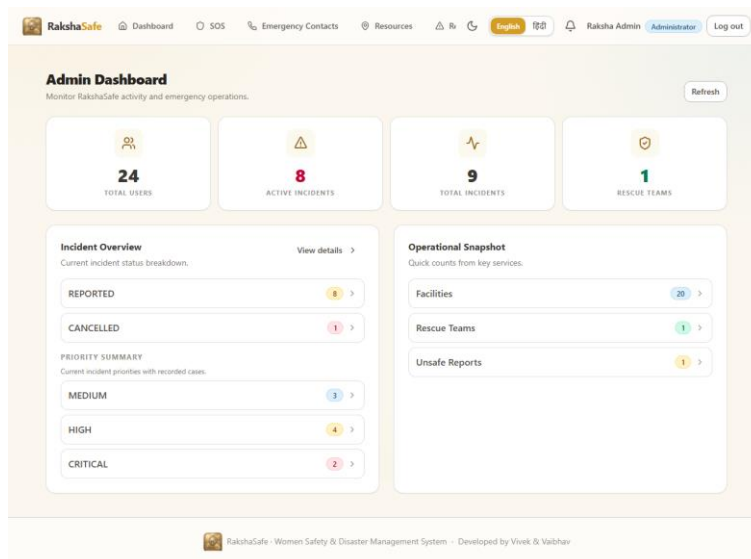
```
export function getStoredToken(): string | null {
  // ... trimmed (101 more lines) ...
}
```

5.3.2 Resources, SOS and Admin Screens

The resources page lists facilities and teams with click-to-call tel links; the SOS page posts the geolocated incident; admin screens are guarded behind the ADMIN role.

Source file: `frontend/src/pages/ResourcesPage.tsx`

```
import { useCallback, useEffect, useRef, useState } from 'react'
import { ApiError, api } from '../lib/api'
import { useI18n } from '../lib/i18n'
import { requestDeviceLocation, type LocationOutcome } from '../lib/geolocation'
import {
  FACILITY_TYPES,
  TEAM_TYPES,
  formatCoords,
  getOsmNearby,
  buildTelHref,
  buildDirectionsUrl,
  type Facility,
  type NearbyResource,
  type RescueTeam,
} from '../lib/resources'
import { NearbyResourcesSection } from '../components/resources/NearbyResourcesSection'
import { Alert } from '../components/ui/Alert'
import { Badge } from '../components/ui/Badge'
// ... trimmed (491 more lines) ...
```



Output screen: administrator login redirecting to `#/admin/dashboard` with role ADMIN (T10).

5.4 Functional Testing and Output Screens

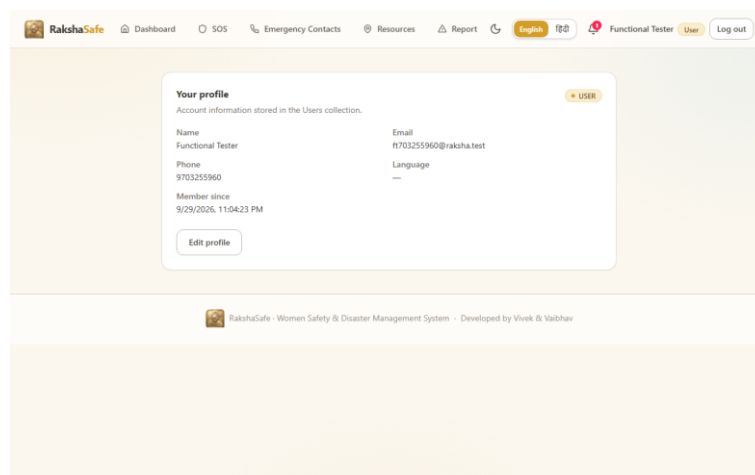
A functional end-to-end suite drives a real Chromium browser against the live services (frontend :5173, backend :5000, AI :8000, real MongoDB). Sixteen scenarios ran; thirteen passed, two genuine defects were captured, and one scenario showed the expected graceful degradation of the AI risk module.

#	Test Case	Result	Evidence
T1	App load	PASS	title ok, 0 console/page errors
T2	Registration	PASS	redirect to dashboard, <code>/auth/me</code> returns user
T3	Session persistence	PASS	token survives reload, logout clears
T4	Dashboard buttons	PASS	all modules open at their route
T5	Emergency contacts CRUD	PASS	create/edit/delete persists
T6	SOS + GPS + address	PASS	28.6139/77.209, status REPORTED

T7	Resources	PASS	facilities + teams, 21 tel links
T8	AI risk assessment	FAIL (expected)	graceful 502, no invented score
T9	Notifications	PASS	list loads, mark-all-read works
T10	Admin login	PASS	#/admin/dashboard, role ADMIN
T11	Admin modules	PASS	8 admin routes open
T12	Admin contacts	PASS	2 rows, edit persists
T13	Role security	PASS	403 for USER on admin APIs, 401 unauth
T14	Responsive overflow	FAIL	390px resources overflow 548px
T15	Language EN-HI	PASS	925 Devanagari chars, persists
T16	Final scan	FAIL	weather 502 x4 (Open-Meteo typo)

5.4.1 Defects Found and Root Cause

T14: the resources grid overflowed 548px at a 390px viewport because grid items kept min-width:auto. T16: four weather 502s came from misspelled Open-Meteo variable names (temperature_2 instead of temperature_2m) in the weather service. Both have a verified, one-line fix.



Output screen: resources at 390px with horizontal overflow measured by the test (T14).

5.5 Conclusion

The implementation follows a clean three-layer architecture and every module shown in this chapter was captured with its real screen output. Thirteen of sixteen functional scenarios pass; the two genuine defects have precise root causes and one-line fixes, and the AI risk module fails gracefully by design. The live screenshots throughout this chapter are the evidence that the system runs as designed.